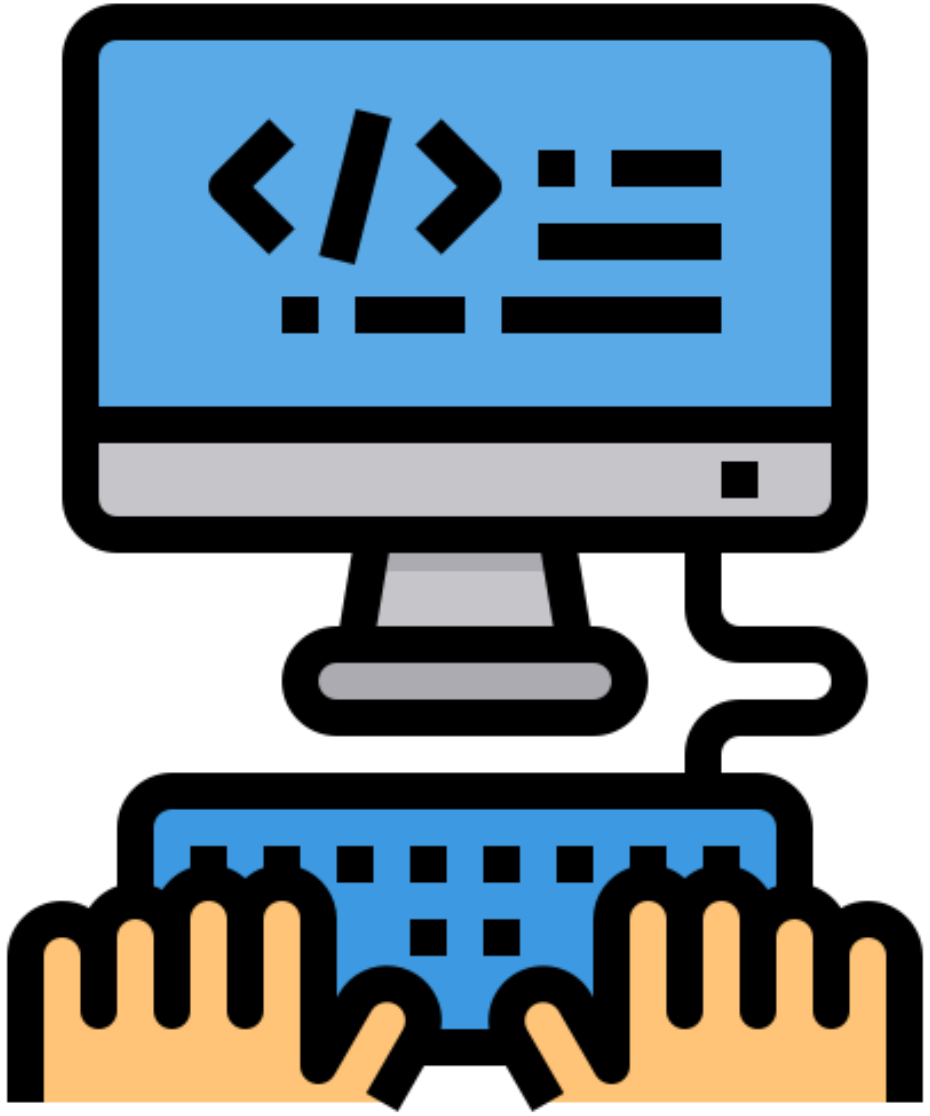
CS 24 AP Computer Science A Review

Week 1: Primitive Data Types

DR. ERIC CHOU
IEEE SENIOR MEMBER

Topics

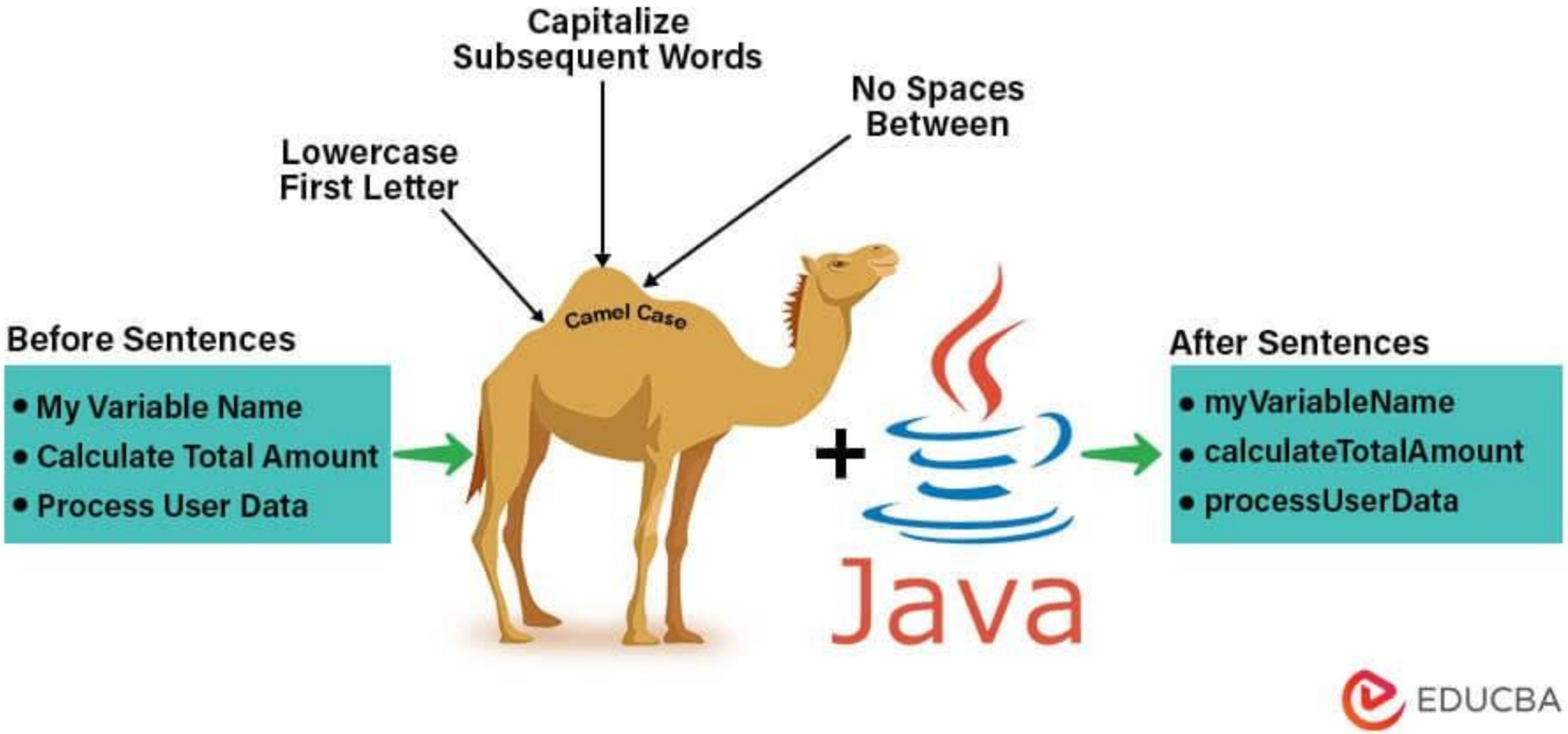
- Program Style
- Compiling and Errors
- Objects and Primitive Data Types
- Primitive Data Types – int, double, char, boolean
- Operators



Program Style

SECTION 1

CAMEL CASE IN JAVA



TalentBattle

Java Comments

1 Single Line Comments

```
//This is single line comment
```

2 Multi Line Comments

```
/*
This
is
multi line
comment
*/
```

3 Documentation Comments

```
/**
*
*We can use various tags to depict the parameter
*or heading or author name
*We can also use HTML tags
*
*/
```

text file named `HelloWorld.java`

name

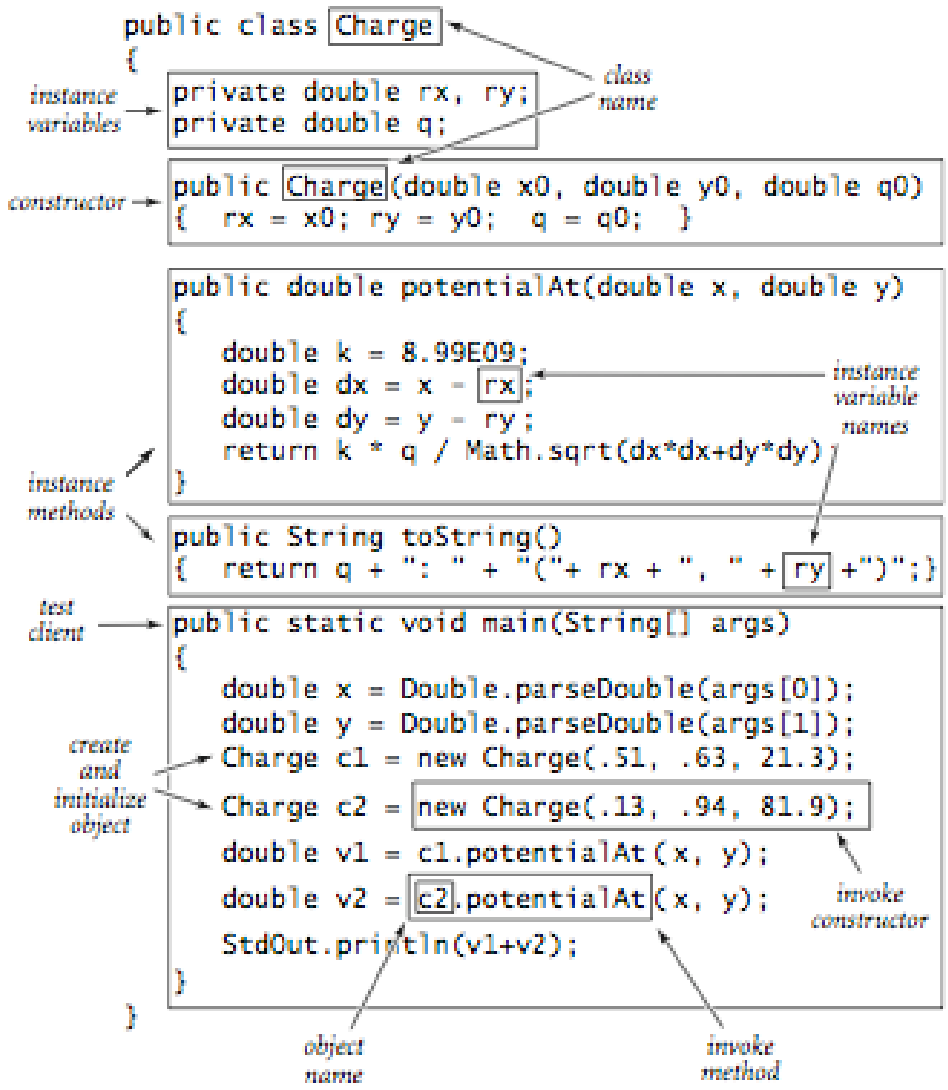
main() method

```
public class HelloWorld
{
   public static void main(String[] args)
   {
      // Prints "Hello, World" in the terminal window.
      System.out.print("Hello, World");
   }
}
```

statements

body

```
class Class_Name{
    final static CONSTANTS;
    static methods(){}
    data fields (instance variables);
    Constructor(){}
    instanceMethods(){}
    overrideMethods(){}
    public static void main(){}
}
```



Anatomy of a class

Multiple Choice

Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `// This is a comment`

Ⓑ `/* This is a comment */`

Ⓒ `// myname is a good identifier name`

Ⓓ all of the above statements will compile.

Multiple Choice

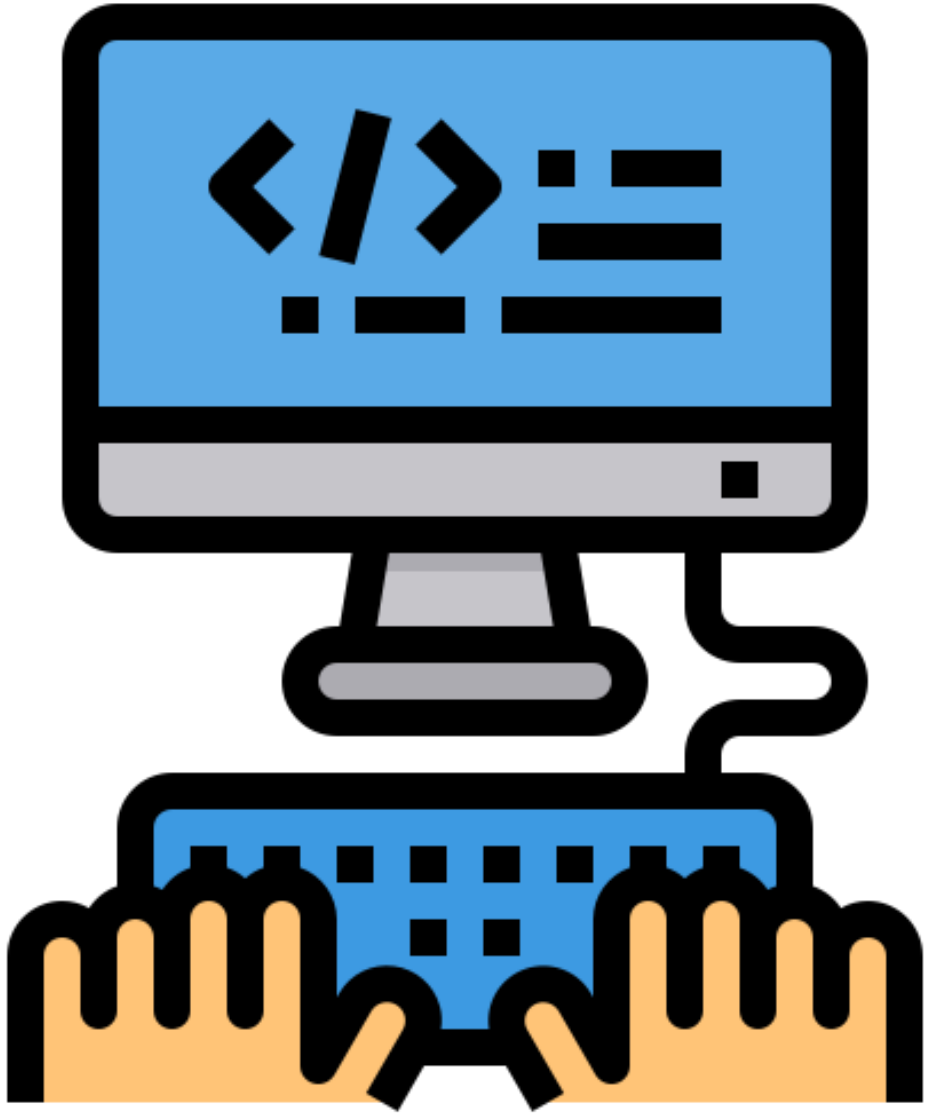
Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `// This is a comment`

Ⓑ `/* This is a comment */`

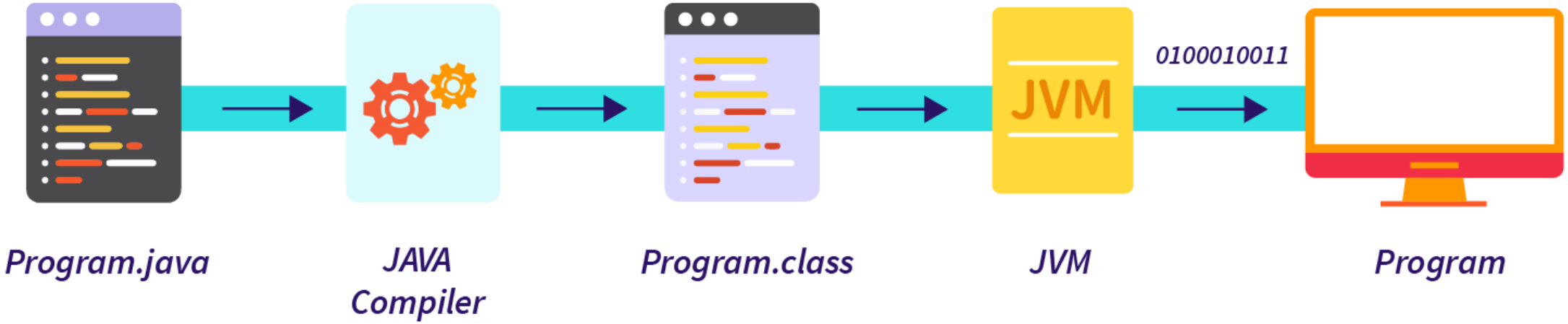
Ⓒ `// myname is a good identifier name`

Ⓓ all of the above statements will compile.



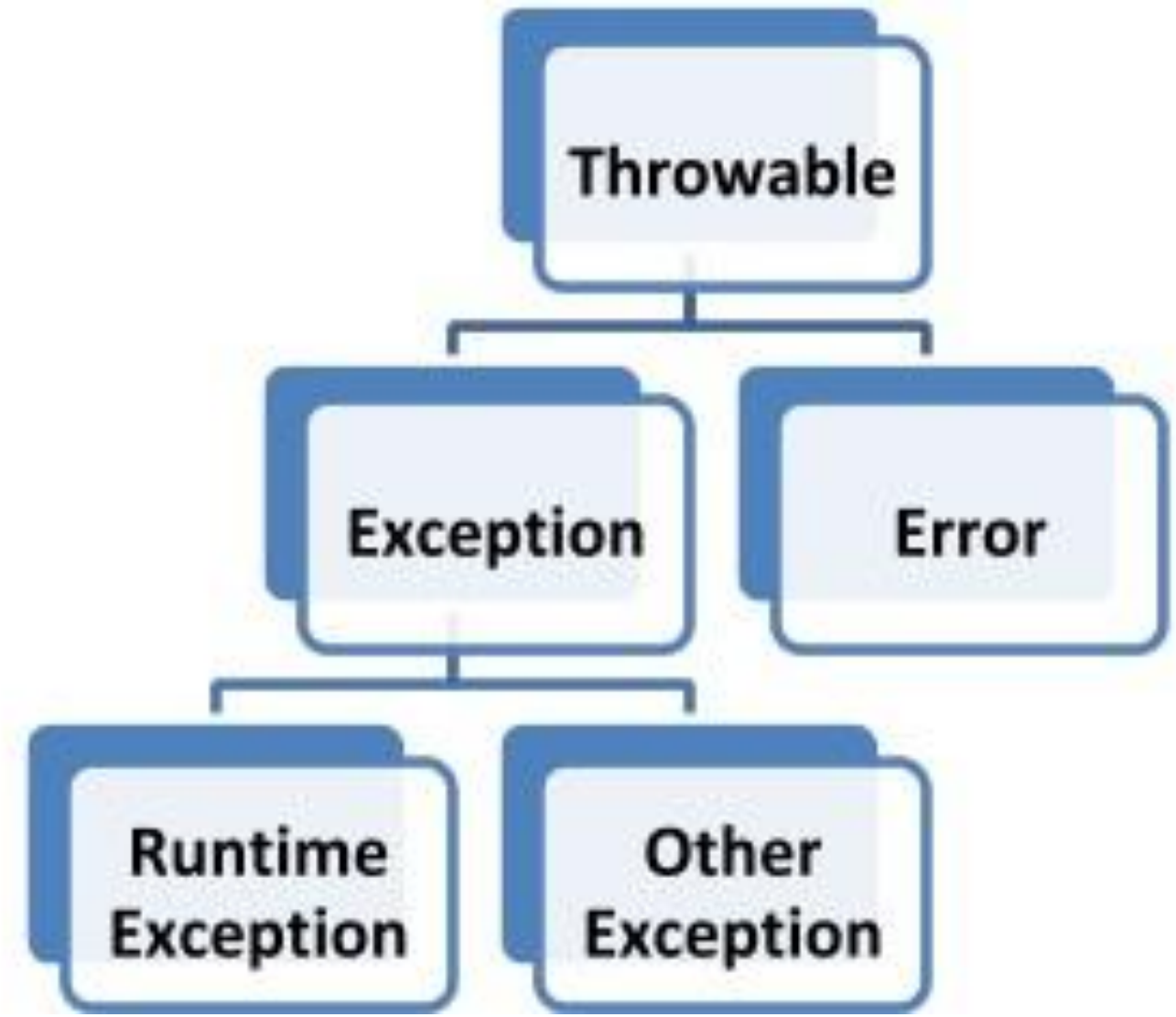
Compiling & Errors

SECTION 2



SCALER Topics

eC Learning Channel


Throwable
Exception
Error
Runtime Exception
Other Exception

```java
try {
    int budget = 1000;
    System.out.println("Success");
}
catch(Exception ex) {
    System.out.println(ex);
}
finally {
    System.out.println("This always runs");
}
```

Errors and Exceptions

- An exception is an error condition that occurs during the execution of a Java program. For example, if you divide an integer by zero, an **ArithmeticException** will be thrown. If you use a negative array index, an **ArrayIndexOutOfBoundsException** will be thrown.
- An unchecked exception is one that is automatically handled by Java's standard exception handling methods, which terminate execution. It is thrown if an attempt is made to divide an integer by 0, or if an array index goes out of bounds, and so on. The exception tells you that you now need to fix your code!

Errors and Exceptions

- A checked exception is one where you provide code to handle the exception, either a try/catch/finally statement, or an explicit throw new ... Exception clause. These exceptions are not necessarily caused by an error in the code. For example, an unexpected end-of-file could be due to a broken network connection.
- Checked exceptions are not part of the AP Java subset.

Exception
ArithmeticException
NullPointerException
ArrayIndexOutOfBoundsException
IndexOutOfBoundsException
IllegalArgumentException
ConcurrentModificationException

The following unchecked exceptions are in the AP Java subset:

Example 1

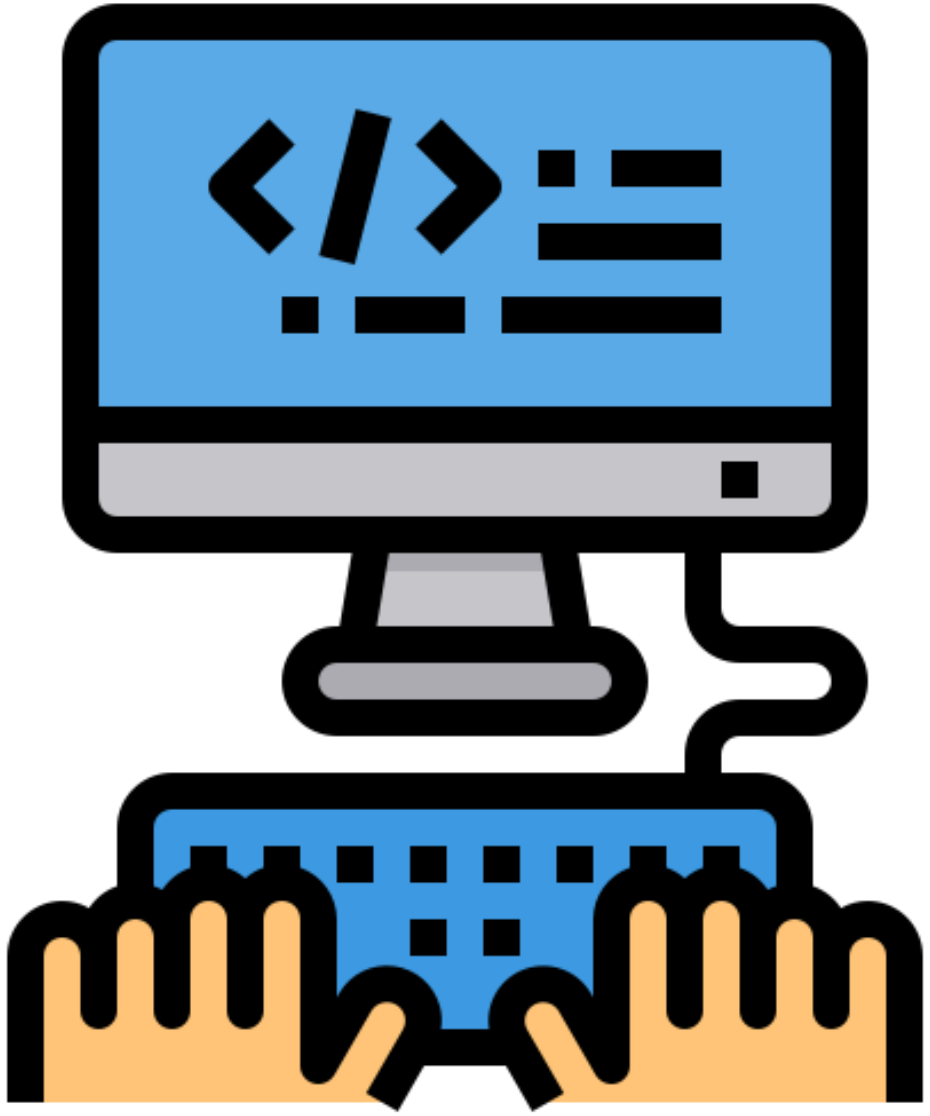
```
if (nurnScores 0)
  throw new ArithmeticException("Cannot divide
  by zero");

else
  findAverageScore();
```

Example 2

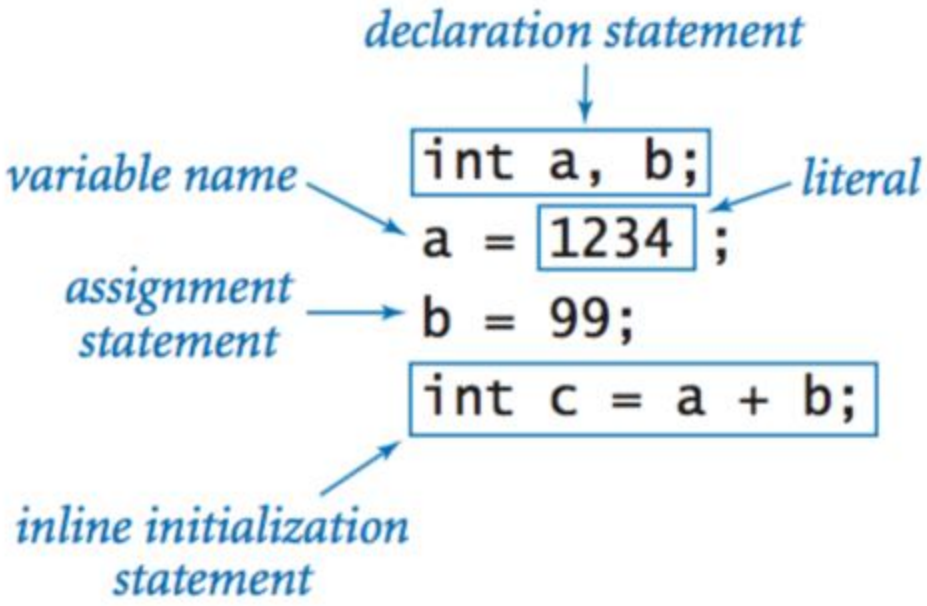
```
public void setRadius(int newRadius) {
   if (newRadius < 0)
    throw new IllegalArgumentException ("Radius cannot
    be negative");
   else
    radius = newRadius;

}
```

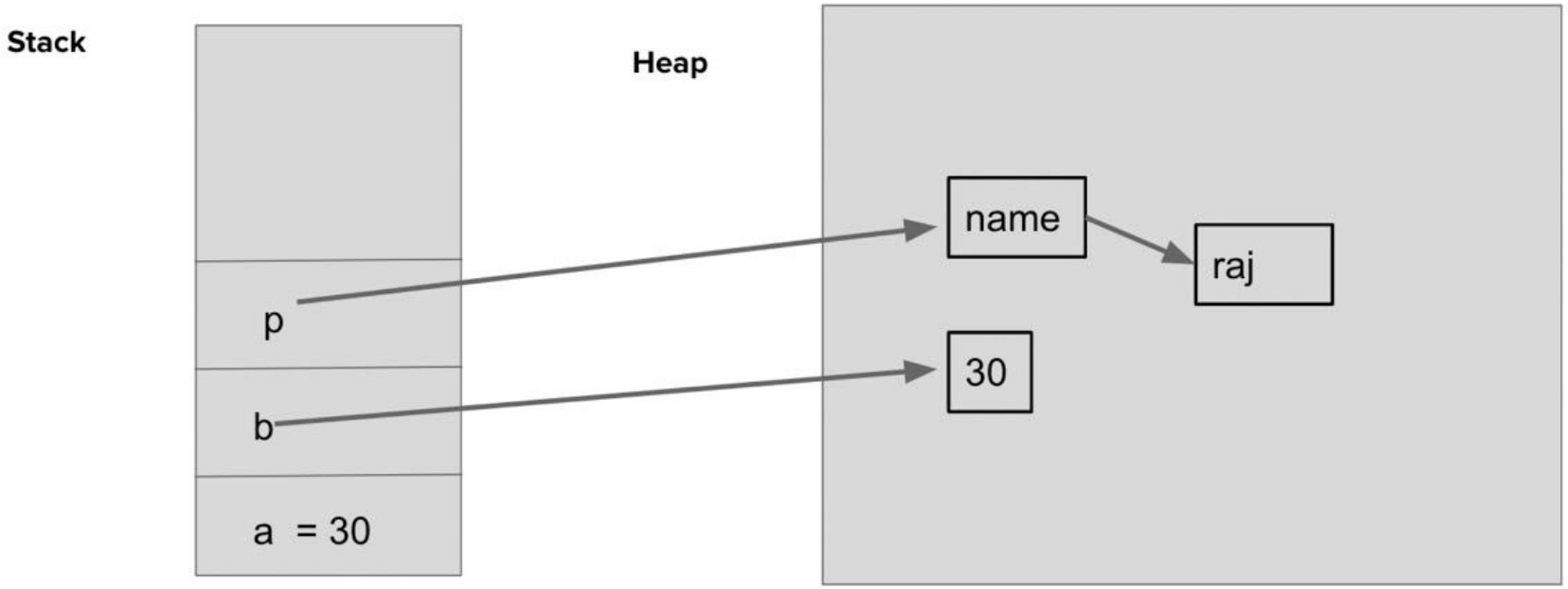


Objects and Primitive Data

SECTION 3

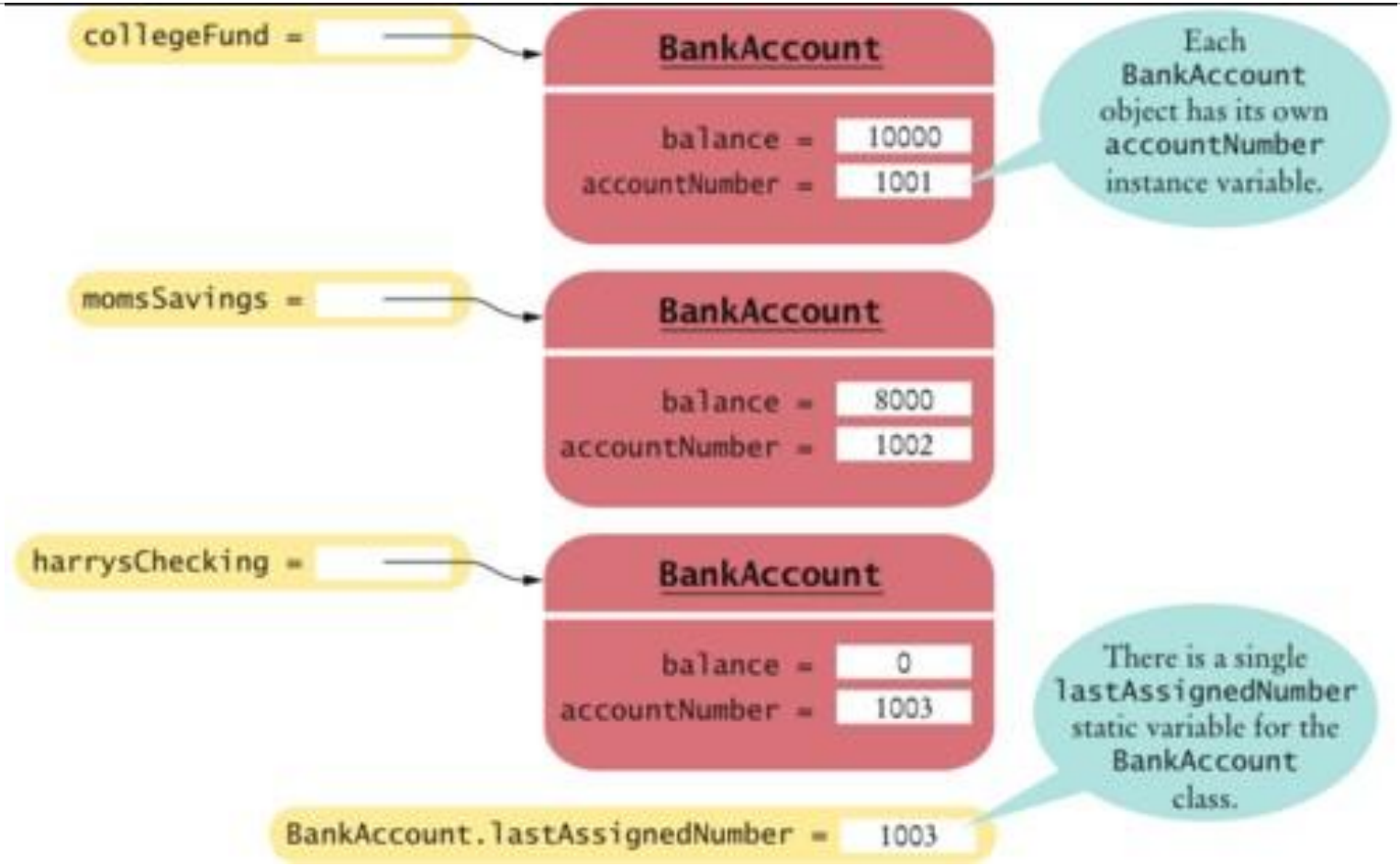

declaration statement
int a, b;
variable name
literal
a = 1234 ;
assignment statement
b = 99;
int c = a + b;
inline initialization statement

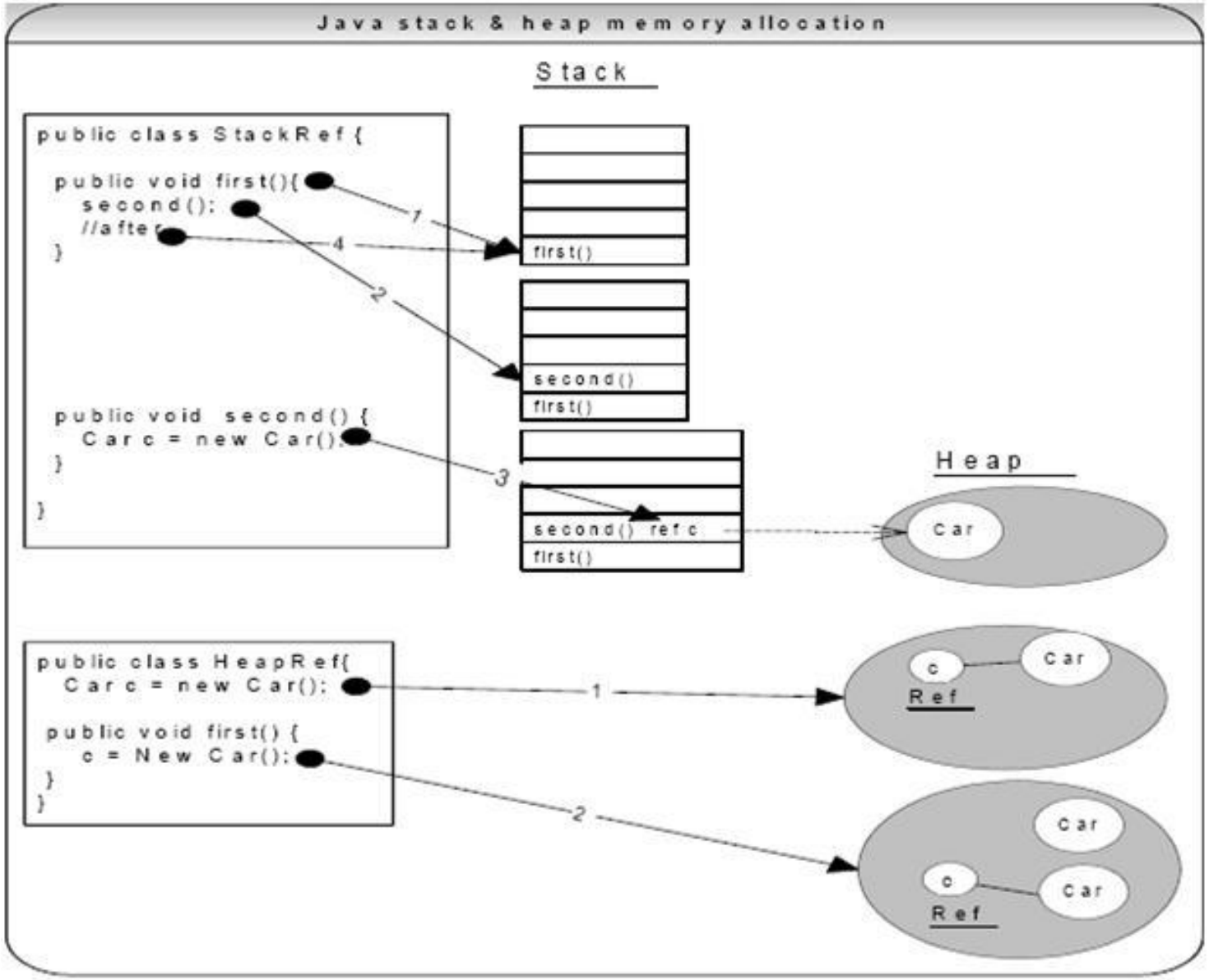
Primitive Vs Reference Data Types In Java

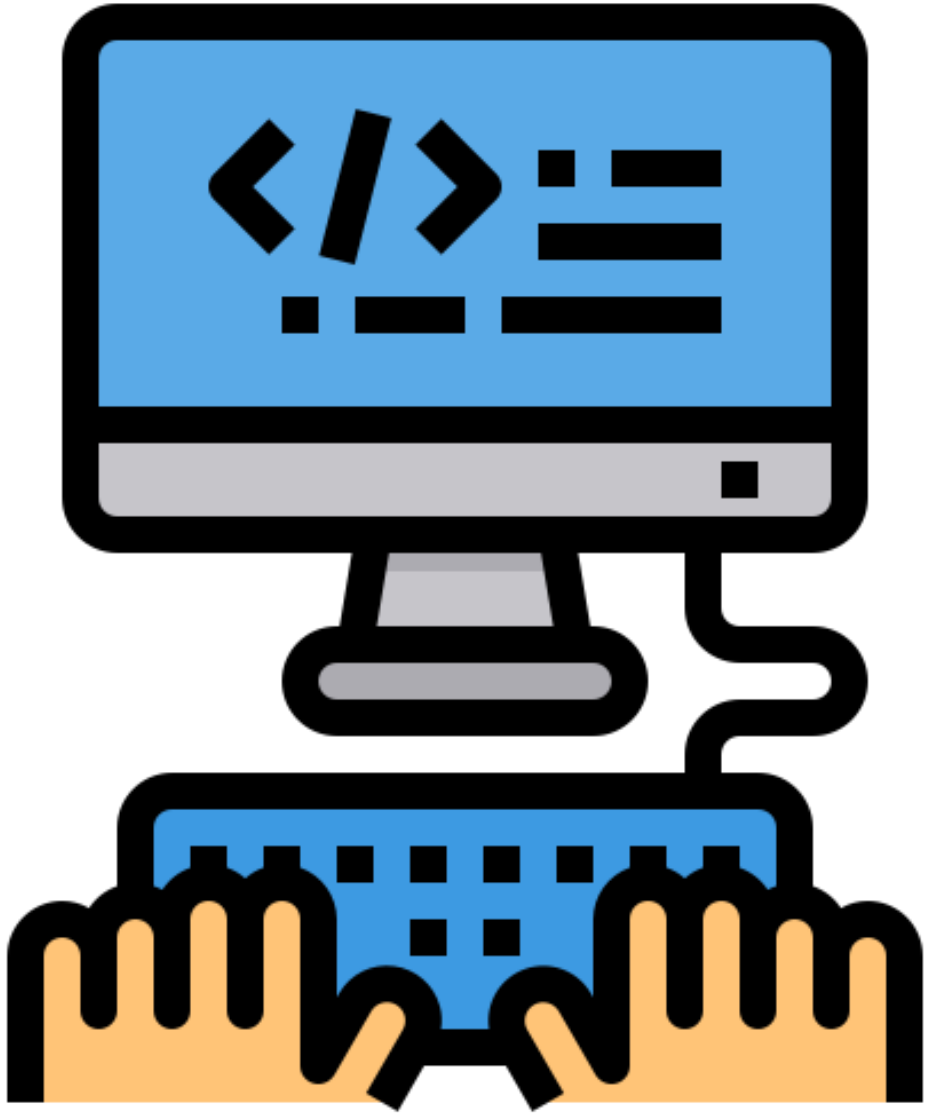


https://webrewrite.com

Static Members/Instance Members




Java stack & heap memory allocation
Stack
public class StackRef {
public void first(){
second();
//after
}
public void second() {
Car c = new Car();
}
}
first()
second()
first()
second() ref c
first()
Heap
Car
public class HeapRef{
Car c = new Car();
public void first() {
c = New Car();
}
}
c
Car
Ref
Car
c
Car
Ref
1
2
3
4



The 4 Data Types – int, double, boolean, char

SECTION 4

type	*set of values*	*common operators*	*sample literal values*
`int`	integers	`+ - * / %`	`99 12 2147483647`
`double`	floating-point numbers	`+ - * /`	`3.14 2.5 6.022e23`
`boolean`	boolean values	`&& || !`	`true false`
`char`	characters		`'A' '1' '%' '\n'`
`String`	sequences of characters	`+`	`"AB" "Hello" "2.5"`

values	integers between -2^{31} and $+2^{31}-1$					
typical literals	1234 99 0 1000000					
operations	*sign*	*add*	*subtract*	*multiply*	*divide*	*remainder*
operators	+ -	+	-	*	/	%

expression	*value*	*comment*
99	99	*integer literal*
+99	99	*positive sign*
-99	-99	*negative sign*
5 + 3	8	*addition*
5 - 3	2	*subtraction*
5 * 3	15	*multiplication*
5 / 3	1	*no fractional part*
5 % 3	2	*remainder*
1 / 0		*run-time error*
3 * 5 - 2	13	** has precedence*
3 + 5 / 2	5	*/ has precedence*
3 - 5 - 2	-4	*left associative*
(3 - 5) - 2	-4	*better style*
3 - (5 - 2)	0	*unambiguous*

values	real numbers (specified by IEEE 754 standard)			
typical literals	`3.14159`	`6.022e23`	`2.0`	`1.4142135623730951`
operations	*add*	*subtract*	*multiply*	*divide*
operators	`+`	`-`	`*`	`/`

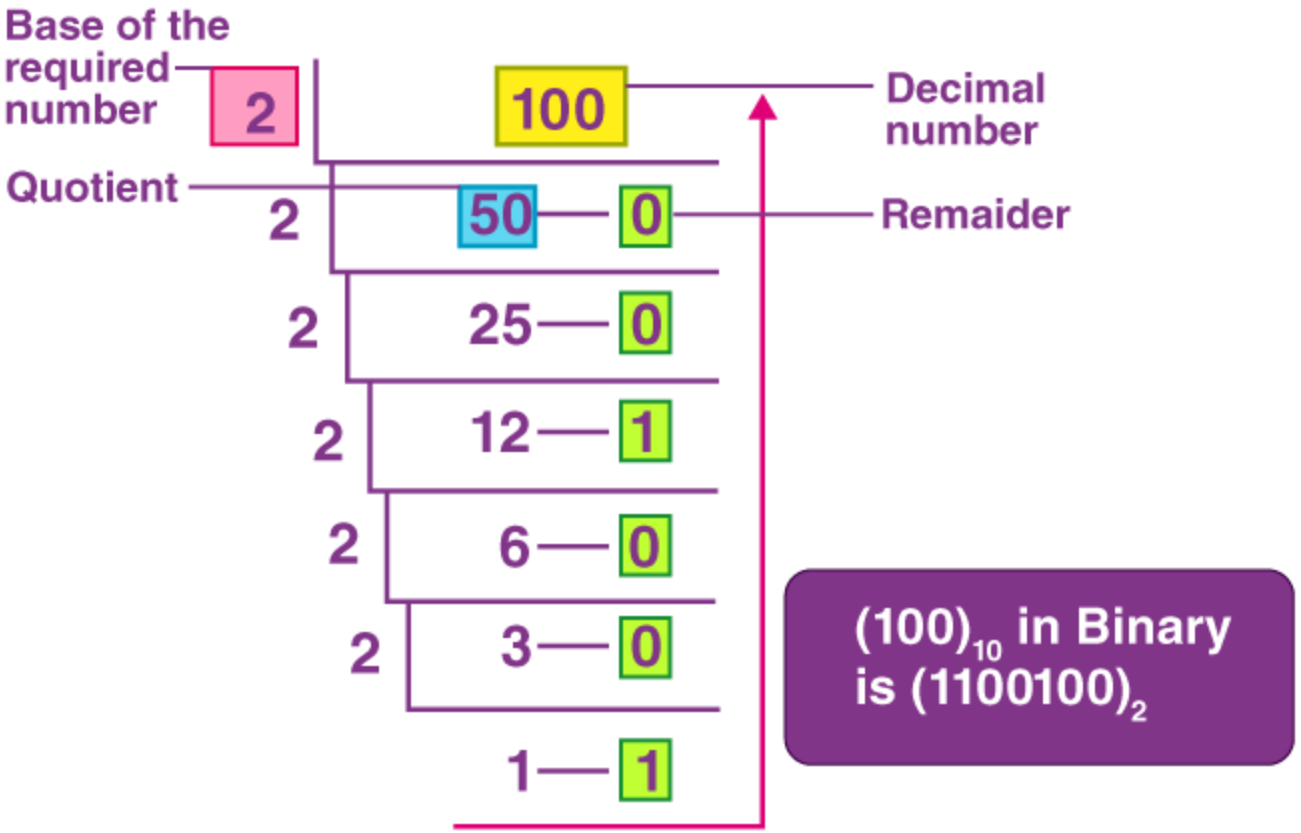
expression	*value*
3.141 + 2.0	5.141
3.141 - 2.0	1.111
3.141 / 2.0	1.5705
5.0 / 3.0	1.6666666666666667
10.0 % 3.141	0.577
1.0 / 0.0	Infinity
Math.sqrt(2.0)	1.4142135623730951
Math.sqrt(-1.0)	NaN

values	*true or false*		
literals	`true`	`false`	
operations	and	or	not
operators	&&	||	!

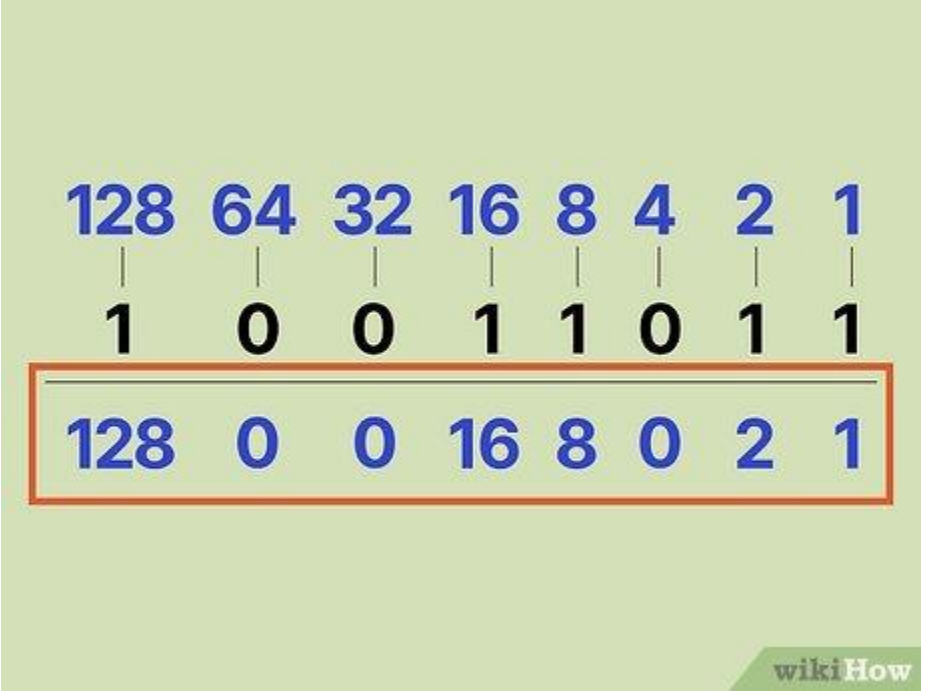
a	*!a*
true	false
false	true

a	*b*	*a && b*	*a || b*
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
	00 0000 0000	01 0000 0001	02 0000 0010	03 0000 0011	04 0000 0100	05 0000 0101	06 0000 0110	07 0000 0111	08 0000 1000	09 0000 1001	10 0000 1010	11 0000 1011	12 0000 1100	13 0000 1101	14 0000 1110	15 0000 1111	
	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI	
0	□	⌜	⊥	⌟	⍀	⊠	✓	⍝	⍲	⍱	≡	⍒	⍗	⍋	⊗	⊙	8
	16 0001 0000	17 0001 0001	18 0001 0010	19 0001 0011	20 0001 0100	21 0001 0101	22 0001 0110	23 0001 0111	24 0001 1000	25 0001 1001	26 0001 1010	27 0001 1011	28 0001 1100	29 0001 1101	30 0001 1110	31 0001 1111	
	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US	
1	⊟	◔	◑	◒	◕	⍻	⎍	⊣	⧖	⍿	⍝	⊖	◰	◱	◲	◳	9
	32 0010 0000	33 0010 0001	34 0010 0010	35 0010 0011	36 0010 0100	37 0010 0101	38 0010 0110	39 0010 0111	40 0010 1000	41 0010 1001	42 0010 1010	43 0010 1011	44 0010 1100	45 0010 1101	46 0010 1110	47 0010 1111	
2	SP	!	"	#	$	%	&	′	(	)	*	+	,	–	.	/	A
	48 0011 0000	49 0011 0001	50 0011 0010	51 0011 0011	52 0011 0100	53 0011 0101	54 0011 0110	55 0011 0111	56 0011 1000	57 0011 1001	58 0011 1010	59 0011 1011	60 0011 1100	61 0011 1101	62 0011 1110	63 0011 1111	
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?	B
	64 0100 0000	65 0100 0001	66 0100 0010	67 0100 0011	68 0100 0100	69 0100 0101	70 0100 0110	71 0100 0111	72 0100 1000	73 0100 1001	74 0100 1010	75 0100 1011	76 0100 1100	77 0100 1101	78 0100 1110	79 0100 1111	
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	C
	80 0101 0000	81 0101 0001	82 0101 0010	83 0101 0011	84 0101 0100	85 0101 0101	86 0101 0110	87 0101 0111	88 0101 1000	89 0101 1001	90 0101 1010	91 0101 1011	92 0101 1100	93 0101 1101	94 0101 1110	95 0101 1111	
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_	D
	96 0110 0000	97 0110 0001	98 0110 0010	99 0110 0011	100 0110 0100	101 0110 0101	102 0110 0110	103 0110 0111	104 0110 1000	105 0110 1001	106 0110 1010	107 0110 1011	108 0110 1100	109 0110 1101	110 0110 1110	111 0110 1111	
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	E
	112 0111 0000	113 0111 0001	114 0111 0010	115 0111 0011	116 0111 0100	117 0111 0101	118 0111 0110	119 0111 0111	120 0111 1000	121 0111 1001	122 0111 1010	123 0111 1011	124 0111 1100	125 0111 1101	126 0111 1110	127 0111 1111	
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL	F



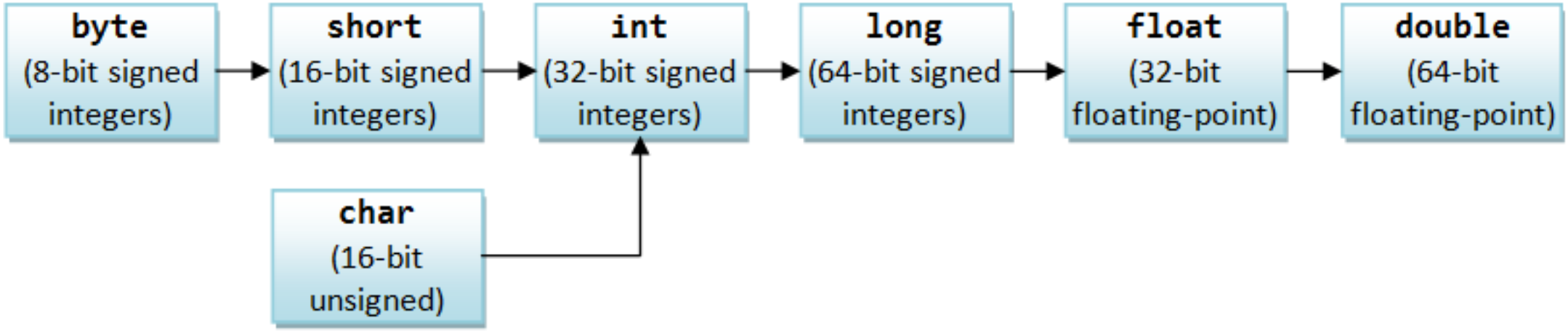
eC Learning Channel


128 64 32 16 8 4 2 1
1 0 0 1 1 0 1 1
128 0 0 16 8 0 2 1
wikiHow

www.allspectrum.com

Binary	Decimal	Hex
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

order more free cards at
www.allspectrum.com/cards



Orders of Implicit Type-Casting for Primitives

Multiple Choice

Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `System.out.print("I love Java");`

Ⓑ `System.out.print(1 + "love" + Java");`

Ⓒ `System.out.print(1 + "love" "Java");`

Ⓓ `System.out.print("I love" + " " + "Java");`

Multiple Choice

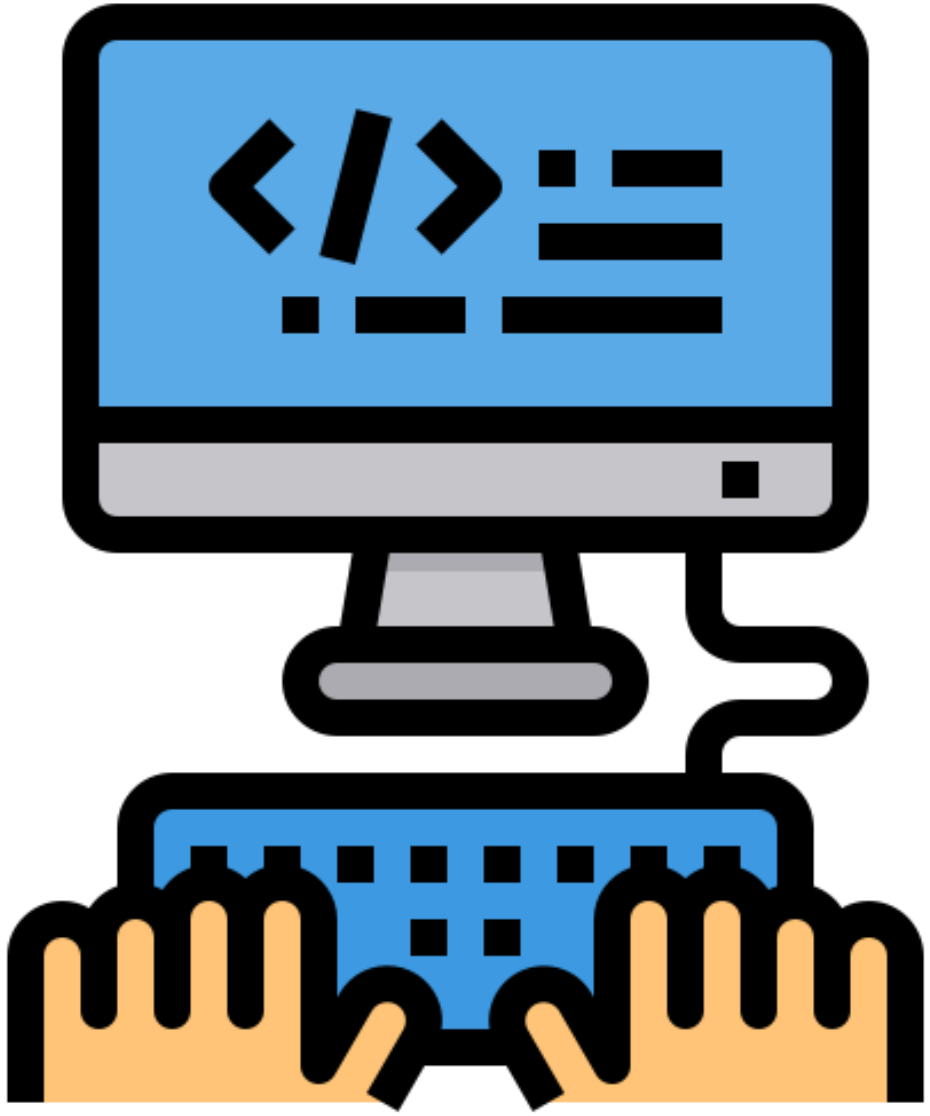
Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `System.out.print("I love Java");`

Ⓑ `System.out.print(1 + "love" + Java");`

Ⓒ `System.out.print(1 + "love" + "Java");`

Ⓓ `System.out.print("I love" + " " + "Java");`



Operators

SECTION 5

Java Arithmetic Operators

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

Integer Division and Its Usage

- int a = b * q + r; // a, b, q, r are all integers
- q = a / b;
- b = a / q;
- r = a % q;
- r = a % b;

Multiple Choice

Consider the following Code Segment:

```
int a = 3;
int b = 6;
int c = 8;
int d = a/b;
c /= d;
System.out.print(c);
```

Multiple Choice

Ⓐ 4

Ⓑ 8

Ⓒ 16

Ⓓ There is no output because of a run-time error.

Multiple Choice

Ⓐ 4

Ⓑ 8

Ⓒ 16

Ⓓ There is no output because of a run-time error.

op	*meaning*	`true`	`false`
`==`	*equal*	`2 == 2`	`2 == 3`
`!=`	*not equal*	`3 != 2`	`2 != 2`
`<`	*less than*	`2 < 13`	`2 < 2`
`<=`	*less than or equal*	`2 <= 2`	`3 <= 2`
`>`	*greater than*	`13 > 2`	`2 > 13`
`>=`	*greater than or equal*	`3 >= 2`	`2 >= 3`

non-negative discriminant?	`(b*b - 4.0*a*c) >= 0.0`
beginning of a century?	`(year % 100) == 0`
legal month?	`(month >= 1) && (month <= 12)`

`void System.out.print(String s)`	*print s*
`void System.out.println(String s)`	*print s, followed by a newline*
`void System.out.println()`	*print a newline*

Multiple Choice

Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `System.out.print(1);`

Ⓑ `System.out.print("1");`

Ⓒ `System.out.print(side1);`

Ⓓ `System.out.print("side1");`

Multiple Choice

Assuming all other statements in the program are correct, each of the following statements will allow the program to compile EXCEPT

Ⓐ `System.out.print(1);`

Ⓑ `System.out.print("1");`

Ⓒ `System.out.print(side1);`

Ⓓ `System.out.print("side1");`

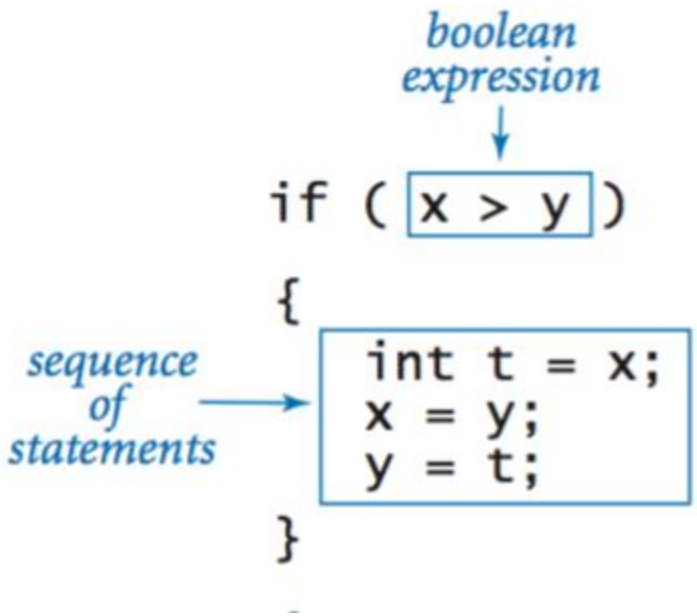
`int`	`Integer.parseInt(String s)`	*convert* `s` *to an* `int` *value*
`double`	`Double.parseDouble(String s)`	*convert* `s` *to a* `double` *value*
`long`	`Long.parseLong(String s)`	*convert* `s` *to a* `long` *value*

public class Math

double abs(double a)	*absolute value of a*
double max(double a, double b)	*maximum of a and b*
double min(double a, double b)	*minimum of a and b*
double sin(double theta)	*sine of theta*
double cos(double theta)	*cosine of theta*
double tan(double theta)	*tangent of theta*
double toRadians(double degrees)	*convert angle from degrees to radians*
double toDegrees(double radians)	*convert angle from radians to degrees*
double exp(double a)	*exponential* (e^a)
double log(double a)	*natural log* ($\log_e a$, *or* $\ln a$)
double pow(double a, double b)	*raise a to the bth power* (a^b)
long round(double a)	*round a to the nearest integer*
double random()	*random number in* [0, 1)
double sqrt(double a)	*square root of a*
double E	*value of e (constant)*
double PI	*value of* π *(constant)*

method call	*library*	*return type*	*value*
`Integer.parseInt("123")`	`Integer`	`int`	123
`Double.parseDouble("1.5")`	`Double`	`double`	1.5
`Math.sqrt(5.0*5.0 - 4.0*4.0)`	`Math`	`double`	3.0
`Math.log(Math.E)`	`Math`	`double`	1.0
`Math.random()`	`Math`	`double`	*random in* $[0, 1)$
`Math.round(3.14159)`	`Math`	`long`	3
`Math.max(1.0, 9.0)`	`Math`	`double`	9.0

expression	*expression type*	*expression value*
`(1 + 2 + 3 + 4) / 4.0`	`double`	`2.5`
`Math.sqrt(4)`	`double`	`2.0`
`"1234" + 99`	`String`	`"123499"`
`11 * 0.25`	`double`	`2.75`
`(int) 11 * 0.25`	`double`	`2.75`
`11 * (int) 0.25`	`int`	`0`
`(int) (11 * 0.25)`	`int`	`2`
`(int) 2.71828`	`int`	`2`
`Math.round(2.71828)`	`long`	`3`
`(int) Math.round(2.71828)`	`int`	`3`
`Integer.parseInt("1234")`	`int`	`1234`


boolean expression
if (x > y)
{
sequence of statements
int t = x;
x = y;
y = t;
}

absolute value	`if (x < 0) x = -x;`

put the smaller value in x and the larger value in y	`if (x > y)`
	`{`
	`  int t = x;`
	`  x = y;`
	`  y = t;`
	`}`

maximum of x and y	`if (x > y) max = x;` `else  max = y;`
error check for division operation	`if (den == 0) System.out.println("Division by zero");` `else  System.out.println("Quotient = " + num/den);`

error check for quadratic formula	`double discriminant = b*b - 4.0*c;`
	`if (discriminant < 0.0)`
	`{`
	`  System.out.println("No real roots");`
	`}`
	`else`
	`{`
	`  System.out.println((-b + Math.sqrt(discriminant))/2.0);`
	`  System.out.println((-b - Math.sqrt(discriminant))/2.0);`
	`}`

```
if      (income <      0) rate = 0.00;
else if (income <   8925) rate = 0.10;
else if (income <  36250) rate = 0.15;
else if (income <  87850) rate = 0.23;
else if (income < 183250) rate = 0.28;
else if (income < 398350) rate = 0.33;
else if (income < 400000) rate = 0.35;
else                      rate = 0.396;
```

initialization is a separate statement

loop-continuation condition

```
int power = 1;
while ( power <= n/2 )
{
  power = 2*power;
}
```

braces are optional when body is a single statement

body

initialize another variable in a separate statement

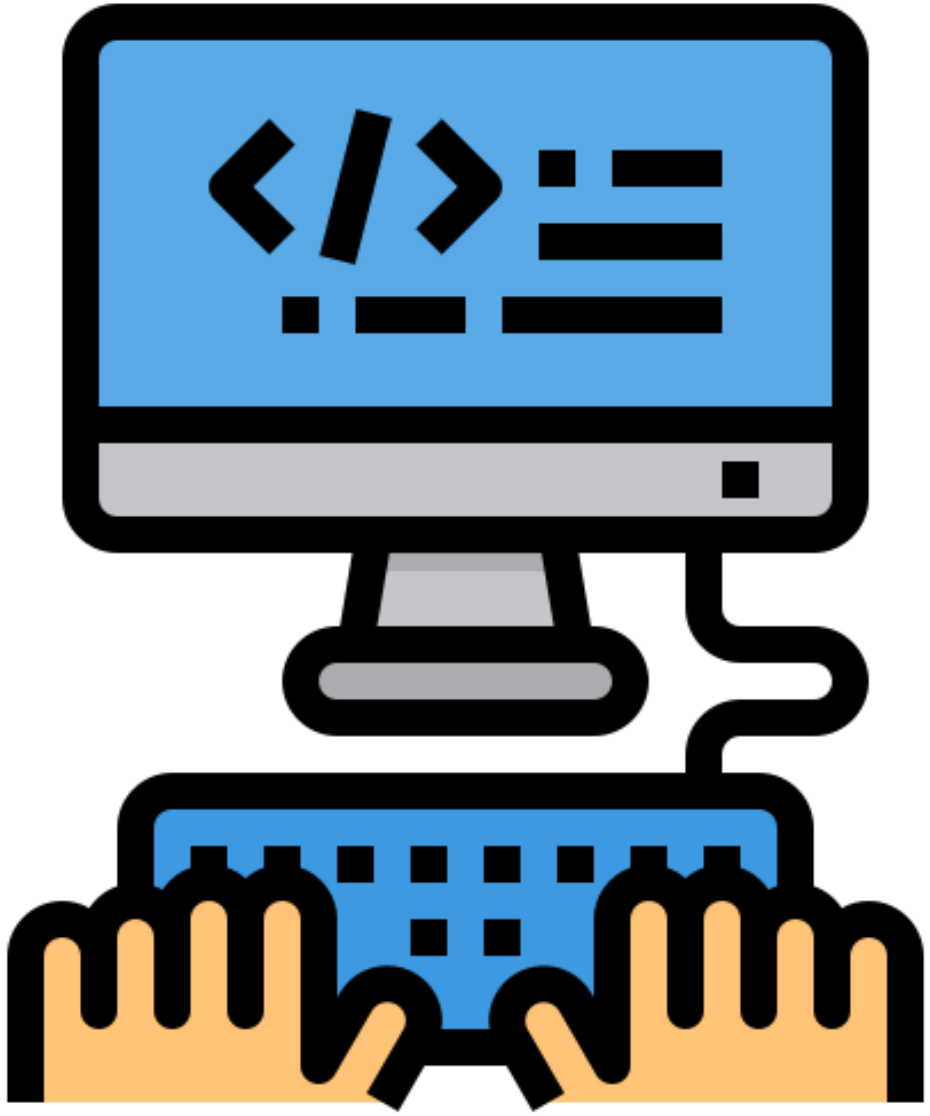
declare and initialize a loop control variable

loop-continuation condition

increment

```
int power = 1;
for (int i = 0; i <= n; i++ )
{
   System.out.println(i + " " + power);
   power = 2*power;
}
```

body



Practice MCQ Problems

SECTION 6

Multiple Choice (1)

Consider the following Code Segment:

```
int a = 10;
double b = 10.7;
double c = a + b;
int d = a + c;
System.out.print(c+” “+d);
```

Multiple Choice (1)

Ⓐ `20.0  30`

Ⓑ `20.7  31`

Ⓒ `20.7  30.7`

Ⓓ Nothing will be printed because of a compile-time error.

Multiple Choice (1)

Ⓐ `20.0  30`

Ⓑ `20.7  31`

Ⓒ `20.7  30.7`

Ⓓ Nothing will be printed because of a compile-time error.

Multiple Choice (2)

Consider the following Code Segment:

```
int a = 10;
double b = 10.7;
int d = a + b;
```

Line 3 will not compile in the code segment above. With which of the following statements could we replace this line so that it completes?

```
Ⅰ. int d = (int) a + b;
Ⅱ. int d = (int) (a+b);
Ⅲ. int d = a + (int) b;
```

Multiple Choice (2)

Ⓐ Ⅱ only

Ⓑ Ⅲ only

Ⓒ Ⅰ and Ⅲ only

Ⓓ Ⅱ and Ⅲ only

Multiple Choice (2)

Ⓐ Ⅱ only

Ⓑ Ⅲ only

Ⓒ Ⅰ and Ⅲ only

Ⓓ Ⅱ and Ⅲ only

Multiple Choice (3)

Consider the following Code Segment:

```
int a = 11;
int b = 4;
double x = 11;
double y = 4;
System.out.print(a/b);
System.out.print(“, ”);
System.out.print(x/y);
System.out.print(“, ”);
System.out.print(a/y);
```

Multiple Choice (3)

Ⓐ 3, 2.75, 2.75

Ⓑ 2, 3, 2

Ⓒ 2, 2.75, 2.75

Ⓓ Nothing will be printed because of a compile-time error.

Multiple Choice (3)

Ⓐ 3, 2.75, 2.75

Ⓑ 2, 3, 2

Ⓒ 2, 2.75, 2.75

Ⓓ Nothing will be printed because of a compile-time error.

Multiple Choice (4)

Consider the following statement:

```
int i = x % 50;
```

If `x` is a positive integer, which of the following could NOT be the value of `i` after the statement above executes?

Ⓐ 0

Ⓑ 10

Ⓒ 25

Ⓓ 50

Multiple Choice (4)

Consider the following statement:

```
int i = x % 50;
```

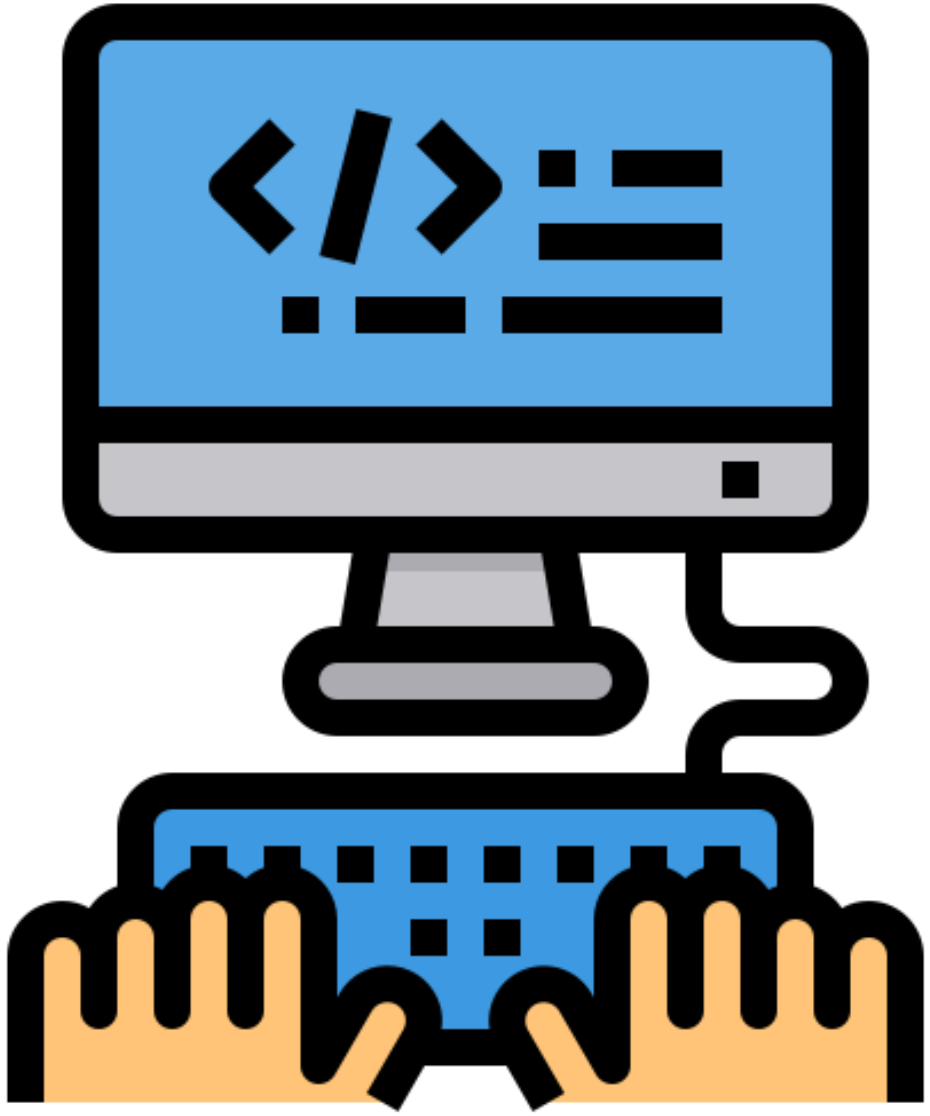
If `x` is a positive integer, which of the following could NOT be the value of `i` after the statement above executes?

Ⓐ 0

Ⓑ 10

Ⓒ 25

Ⓓ 50



FRQ Algorithms

SECTION 7

Algorithm 1: Successive Division for all Digits

```
public int sumOfDigits(int n){
    int s = 0;
    while (n>0){
        int d = n % 10;
        s += d;
        n /= 10;
    }
    return s;
}
```

Algorithm 2: Carry-in and Carry-out

```
// t is an two tuple array t[0] for hour, t[1]
// for minutes, same for s
public int[] hour_minute(int[] t, int[] s){
    int[] u = new int[2];
    int m = t[0]*60+ s[0]*60+t[1]+ s[1];
    u[0] = m/60;
    u[1] = m%60;
    return u;
}
```